

EPF Study Group #3: zkEVMs

Cody Gunton and Ignacio Hagopian - March 25, 2026

<https://codygunton.github.io/talks-and-writing/2026-03-25-epf-study-zkevms/>



zkEVM project 1

Execution Witness

March 2026





engine_newPayloadWithWitness

- Formal specs for optimized *engine_newPayloadWithWitness*
- EL pipeline optimizations included on top of previous SSZ+HTTP version
- CL PoV latency ~1.3s for worst-case scenario
- Links:
 - PR in [ethereum/execution-api](#)
 - [Ethrex implementation branch](#) (WIP)

Execution Witness specs & tests

- Latest release: [zkevm@v0.3.3](#)
- Tests:
 - 139 new tests covering a wide range of edge cases for all execution witness fields
 - Still planned to add more observable failure effects (i.e. OOG, stack overflows, etc)
- Releases on top of latest BAL-devnet tests release.



kev



som



figtracer



ivan

Execution Witness – ELs

- Ethrex
 - Passing tests at zkevm@v0.3.2
- Reth
 - Started to more formally build running environment on top of bal-devnet-3 branch
- Geth
 - PRs being evaluated to run tests releases properly
- Zilkworm
 - [Correctness of execution witness](#) verification for Zilkworm.

zkEVM project 2

Guest Program

Feb 2026



Guest program - Specs & tests

- Latest release: [zkevm@v0.3.3](#)
- Tests:
 - Tests covering correct guest program behavior in invalid inputs
 - 25 new tests were created covering invalid execution witness scenarios
 - Still planned to add more observable failure effects (i.e. OOG, stack overflows, etc)
- Releases on top of latest BAL-devnet tests release.

zkEVM project 6

Benchmarking and Metrics

Feb 2026





New libssz library

- New [libssz](#) library (Rust)
 - *no_std* & highly performant
 - Lambdaclass [Blogpost](#)
- Now [integrated in ere-quests](#)
 - ~2x speedup in public input calculation $\Rightarrow$ ~5% speedup in block proving time
 - Comply with *no_std* requirement from standards

Apple M3 Max (ARM)

Encode

Type	libssz	Lighthouse	ssz_rs	vs Lighthouse	vs ssz_rs
bool	214 ps	3.9 ns	29 ns	18x	135x
u64	235 ps	4.0 ns	29 ns	17x	123x
[u8; 32]	4.1 ns	4.2 ns	30 ns	~1x	7.3x
BeaconBlockHeader	13.7 ns	113 ns	1.8 μ s	8.2x	131x
Vec<u64> (1K)	118 ns	433 ns	14 μ s	3.7x	119x
Vec<u64> (100K)	10.4 μ s	56 μ s	1.5 ms	5.4x	144x

Decode

Type	libssz	Lighthouse	ssz_rs	vs Lighthouse	vs ssz_rs
bool	430 ps	430 ps	432 ps	~1x	~1x
u64	461 ps	461 ps	480 ps	~1x	~1x
[u8; 32]	4.1 ns	3.8 ns	66 ns	~1x	16x
BeaconBlockHeader	12.7 ns	12.3 ns	207 ns	~1x	16x
Vec<u64> (1K)	123 ns	1.23 μ s	780 ns	10x	6.3x
Vec<u64> (100K)	10.3 μ s	154 μ s	112 μ s	15x	10.9x

BeaconState (21 fields, variable-length)

Benchmark	Validators	libssz	Lighthouse	ssz_rs	vs Lighthouse	vs ssz_rs
Encode	16K	808 μ s	756 μ s	75.1 ms	~1x	93x
Encode	100K	654 μ s	5.61 ms	215 ms	8.6x	329x
Encode	300K	11.9 ms	18.0 ms	551 ms	1.5x	46x
Encode	1M	5.67 ms	19.0 ms	1.73 s	3.4x	305x
Decode	16K	123 μ s	237 μ s	9.16 ms	1.9x	74x
Decode	100K	539 μ s	804 μ s	28.4 ms	1.5x	53x
Decode	300K	1.51 ms	2.23 ms	76.1 ms	1.5x	50x
Decode	1M	4.94 ms	7.22 ms	230 ms	1.5x	47x



Ziskemu profiling (Zisk v0.16.1)

- [Ziskemu proving cost re-profiling](#)
- ~2x cost improvement compared to v0.14.0
- Keccak and input costs decreased significantly

Reth (v1.11.0)

Total avg. cost: 29.5B

```
READ_INPUT (0.0%, 13.7K)
DESERIALIZE_INPUT (0.9%, 274.3M)
```

```
EIP-8025 (3.2%)
├─ NEW_PAYLOAD_REQUEST_ROOT (2.5%, 739.2M)
├─ MISC_PREPARATION (0.0%, 2.4M)
└─ NEW_PAYLOAD_REQUEST_TO_BLOCK (0.7%, 204.4M)
```

```
STF (94.5%, 28.0B)
├─ RECOVER_PUBLIC_KEYS (2.9%, 835.6M)
├─ VALIDATE_ANCESTOR_HEADERS (0.0%, 3.5M)
├─ VALIDATE_BLOCK_CONSENSUS (0.7%, 197.8M)
├─ VALIDATE_ACCOUNT_TRIE (33.5%, 9.8B)
├─ EXECUTE_BLOCK (44.8%, 13.5B)
├─ POST_VALIDATION_CHECKS (1.7%, 497.6M)
└─ COMPUTE_POST_STATE_ROOT (10.1%, 2.9B)
```

```
SERIALIZE_OUTPUT (0.0%, 69.2K)
SHA256_OUTPUT_BYTES (0.0%, 505.5K)
WRITE_OUTPUT (0.0%, 24.2K)
```

Ethrex (33695e)

Total avg. cost: 33.4B

```
READ_INPUT (0.0%, 13.9K)
DESERIALIZE_INPUT (13.9%, 4.5B)
```

```
EIP-8025 (3.1%)
├─ NEW_PAYLOAD_REQUEST_ROOT (2.1%, 684.7M)
├─ MISC_PREPARATION (0.0%, 17.8K)
└─ NEW_PAYLOAD_REQUEST_TO_BLOCK (1.0%, 327.8M)
```

```
STF (81.8%, 27.6B)
├─ PRE_STATE_VERIFICATION (0.0%, 102.9K)
├─ PRE_STATE_INIT (24.8%, 8.1B)
├─ ANCESTOR_VALIDATION (0.0%, 6.0M)
├─ VALIDATE_BLOCK_CONSENSUS (0.6%, 193.8M)
├─ SETUP_EVM (0.0%, 11.2K)
├─ EXECUTE_BLOCK (44.7%, 15.4B)
├─ GET_STATE_TRANSITIONS (0.3%, 97.7M)
├─ APPLY_ACCOUNT_UPDATES (5.6%, 1.8B)
├─ POST_VALIDATION_CHECKS (1.6%, 537.5M)
└─ POST_STATE_ROOT_CALCULATION (4.0%, 1.3B)
```

```
SERIALIZE_OUTPUT (0.0%, 30.3K)
SHA256_OUTPUT_BYTES (0.0%, 509.6K)
WRITE_OUTPUT (0.0%, 24.9K)
```



Experimental G* repricing pipeline integration

- Re-use G* repricings methodology from Maria
- Early integration and more upcoming collaboration

← RIG Repricing Analysis Light mode

Data-driven gas cost adjustments with per-opcode diagnostics

Proposal Runtime Estimation Glue Opcodes

[new_gas.csv](#) [results.csv](#) [glue_results.csv](#) [gas_bench_data.csv](#) [glue_opcodes_by_test.csv](#)

New gas cost proposal for EIP-zkevm

This is an automated report generated from the script `./src/estimate_zkevm_repricings.py`. The script uses the runtime estimation output generated by this script. The report with the runtime estimation results can be found in `./reports/eip-zkevm/2026-03-15_2026-03-21/runtime_estimation_autogenerated_report.md`.

Methodology

New gas costs are calculated using an anchor rate of 12 million gas per second, which represents a target execution rate for EVM operations. The formula used is:

$$\text{new_gas} = (\text{anchor_rate} * \text{runtime_ms}) / 1000$$
